

AstroCopilot

Copiloto Conversacional para Missões Espaciais

Global Solution 2026.1 — FIAP · 2º ano de Inteligência Artificial (2TIAO)

Integrante	RM	Frente
Felipe Sabino da Silva	RM563569	F1 — Agente LLM + RAG + Scraping
Juan Felipe Voltolini	RM562890	F2 — NLP & Voz
Luiz Henrique Ribeiro de Oliveira	RM563077	F4 — IoT ESP32 + Edge + ML
Marco Aurélio Eberhardt Assumpção	RM563348	F5 — Backend / Dashboard / DevOps
Paulo Henrique Senise	RM565781	F3 — Visão Computacional

Grupo 42 · Turma 2TIAO — 2º ano de Inteligência Artificial.

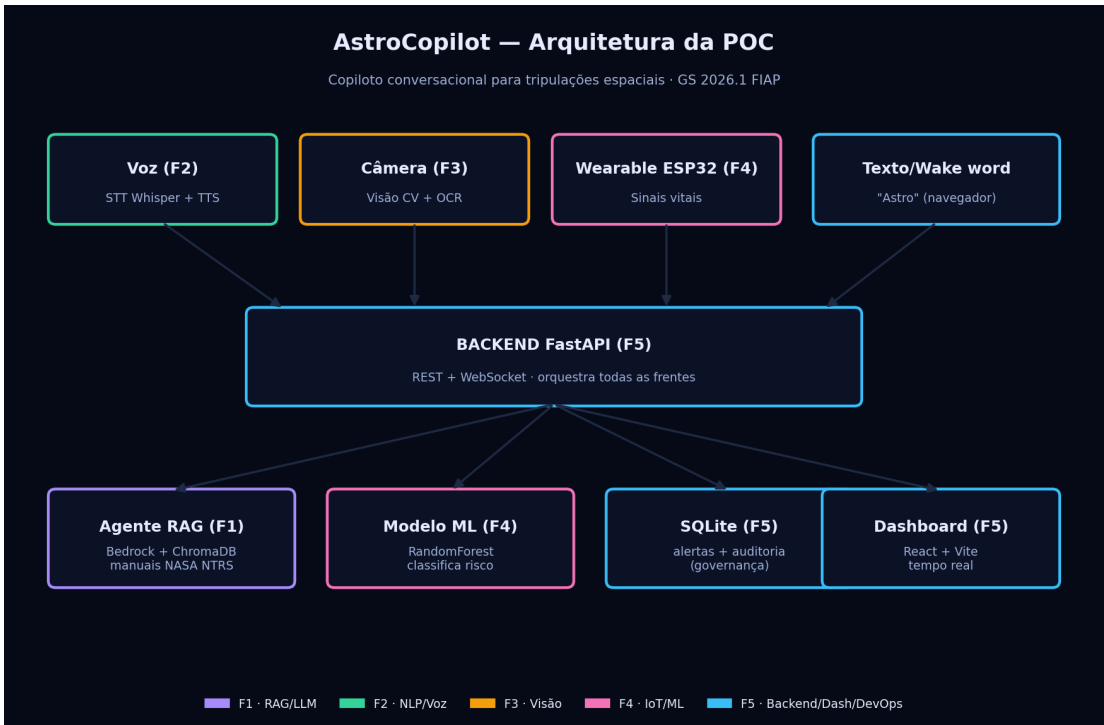


Figura 1 — Arquitetura da solução (5 frentes integradas).

1. Introdução

A pergunta-desafio da GS 2026.1 é: *"Como tecnologias avançadas de Inteligência Artificial, automação e computação podem impulsionar soluções inovadoras para a nova economia espacial?"* A nossa resposta é o **AstroCopilot**: um copiloto de bordo conversacional para tripulações espaciais, que reúne em uma única plataforma a consulta a manuais técnicos reais, o monitoramento de sinais vitais em tempo real, a análise de painéis por imagem e a interação por voz — pensado inclusive em acessibilidade para tripulantes com limitação visual.

Em missões espaciais a tripulação precisa de respostas rápidas, confiáveis e rastreáveis, muitas vezes sem poder folhear manuais ou desviar a atenção dos instrumentos. O AstroCopilot ataca esse problema combinando **IA Generativa com RAG** (respostas fundamentadas em documentos reais da NASA, com citação de fonte), **visão computacional** (leitura de painéis), **IoT com ESP32 e Machine Learning na borda** (telemetria vital classificada por risco) e **voz** (falar e ouvir), tudo orquestrado por um backend e exibido em um dashboard de centro de controle.

A solução foi construída como uma POC integrada por **cinco frentes de trabalho**, uma por integrante, todas convergindo para um backend orquestrador comum. Esta organização permitiu trabalho paralelo desde o primeiro dia (contra contratos de API fixos) e evidencia a **integração interdisciplinar** exigida pelo desafio.

2. Desenvolvimento

2.1 Visão geral da arquitetura

O **backend FastAPI** (Frente 5) é o orquestrador central: expõe uma API REST + WebSocket e chama cada módulo das demais frentes. O **dashboard React + Vite** consome essa API por REST (chat, visão, alertas, auditoria) e por WebSocket (telemetria ao vivo, 1 Hz). A persistência fica em SQLite (histórico de alertas e trilha de auditoria do agente).

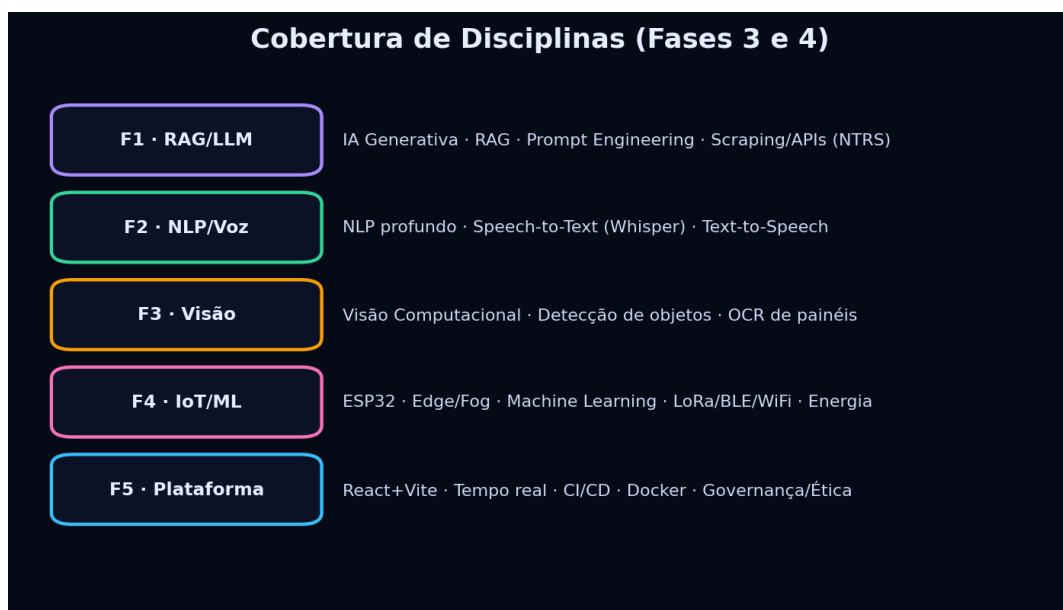


Figura 2 — Disciplinas das Fases 3 e 4 cobertas por frente.

2.2 Contratos de API (integração entre as frentes)

Todas as frentes programaram contra estes contratos. O backend respondia com *mocks* desde o início, permitindo desenvolvimento paralelo; depois cada mock foi substituído pela lógica real da frente correspondente.

Método	Rota	Frente / Função
POST	/api/agent/query	F1 — RAG sobre manuais NASA (cita fonte)
POST	/api/voice	F2 — STT → RAG → TTS + intenção
POST	/api/vision	F3 — detecção de componentes + OCR
POST/GET	/api/telemetry	F4 — telemetria do ESP32 (JSON e query)
WS	/ws/telemetry	F4/F5 — stream de telemetria 1 Hz
GET	/api/alerts	F5 — histórico de risco (SQLite)
GET	/api/audit	F5 — trilha de auditoria do agente

2.3 Frente 1 — Agente LLM + RAG + Scraping

O agente roda sobre **Claude Haiku 4.5 no Amazon Bedrock** (via framework Strands) e tem uma ferramenta `buscar_documentos` que recupera trechos dos manuais indexados no **ChromaDB**. Os documentos vêm de *scraping* da API pública do **NASA NTRS** (Technical Reports Server); os embeddings usam o **Amazon Titan v2**. O *system prompt* aplica Prompt Engineering para respostas curtas e com fonte citada.

`agent-rag/agent.py`

```
def query(text: str, channel: str = "text") -> dict:
    """Responde uma pergunta usando RAG. Retorna {'answer': str, 'sources': list[str]}."""

    channel: 'text' | 'voice' — em voz as respostas ficam ainda mais curtas.
    """
    if not config.configurado():
        raise RuntimeError("RAG nao configurado (API key ou base vetorial ausente).")

    prompt = SYSTEM_PROMPT
    if (channel or "text").lower() == "voice":
        prompt += VOICE_PROMPT_ADDENDUM

    fontes: list[str] = []

    @tool
    def buscar_documentos(consulta: str) -> str:
        """Busca trechos dos manuais espaciais relevantes para a consulta."""
        resultado = _colecao().query(
            query_embeddings=[embeddings.embed(consulta)],
            n_results=config.TOP_K,
        )
        documentos = resultado.get("documents")
        # ... (continua no repositório)
```

2.4 Frente 2 — NLP & Voz

O pipeline de voz transcreve o áudio com **Whisper** (STT), classifica a **intenção** (pergunta/status/emergência), consulta o mesmo agente RAG e sintetiza a resposta em áudio com **Edge-TTS/gTTS**. No dashboard, a ativação por voz usa a *wake word* "Astro".

`voice-nlp/pipeline.py`

```
def process_voice(
    audio_bytes: bytes,
    filename: str | None,
    ask_agent: AskAgent,
    voice_profile: str | None = None,
) -> dict[str, Any]:
    """
    Executa o ciclo completo de voz.
```

```

Retorna:
    transcript, intent, answer_text, sources, answer_audio_file (Path | None)
"""
transcript = stt.transcribe(audio_bytes, filename=filename)
classificacao = intent.classify(transcript)

resultado = ask_agent(transcript)
answer_text = (resultado.get("answer") or "").strip()
sources = resultado.get("sources") or []

audio_path = None
if answer_text:
    try:
        audio_path = tts.synthesize(answer_text, voice_profile=voice_profile)
    except Exception as exc:
        print(f"[Frente 2] TTS falhou (resposta so em texto): {exc}")

return {
    "transcript": transcript,
    "intent": classificacao,
    "answer_text": answer_text,
    "sources": sources,
    "answer_audio_file": audio_path,
}

```

2.5 Frente 3 — Visão Computacional

A imagem enviada pela tripulação passa por **detecção de objetos (YOLOv8)** e por **OCR (Tesseract)** para ler números e alertas do painel. O pipeline devolve o contrato {objects, ocr_text, description}, com tratamento de erro robusto.

vision/pipeline.py

```

"""
Processa os bytes vindos do UploadFile da rota FastAPI.
"""
try:
    # Converte bytes brutos em uma imagem PIL de forma segura
    image = Image.open(io.BytesIO(image_bytes)).convert("RGB")
except Exception as e:
    return {
        "objects": [],
        "ocr_text": "",
        "description": f"Erro crítico ao ler o arquivo de imagem: {e}"
    }

# Converte para array numpy apenas para o detector YOLO
img_array = np.array(image)

# 1. Executa a detecção de objetos (YOLOv8)
try:
    detections = self.detector.detect(img_array)
    # Garante que 'detections' seja uma lista válida, mesmo se vier nula
    if not detections:
        detections = []
except Exception:
    detections = []

# Extrai as classes detectadas de forma segura
objects = [d.get("class", "desconhecido") for d in detections if isinstance(d, dict) and "class" in d]

# 2. OCR: lê o PAINEL INTEIRO primeiro (melhor para displays com vários

```



Figura 3 — Visão: OCR lê as leituras do painel e detecta a anomalia (ALERTA) automaticamente.

2.6 Frente 4 — IoT ESP32 + Edge + Machine Learning

Um wearable **ESP32 (simulado no Wokwi)** lê sensores reais (MPU6050, DS18B20) e envia a telemetria por WiFi/HTTP ao backend. O risco (normal/fadiga/risco) é classificado por um modelo **RandomForest (scikit-learn)** treinado na borda, com *fallback* para regras determinísticas — a API nunca quebra se o modelo não estiver presente.

backend/main.py

```
def classify_risk(hr: float, spo2: float, temp: float,
                 radiation: float = 0.0, resp: float = 14.0) -> str:
    """Classifica o risco do tripulante em normal/fadiga/risco.

    Usa o modelo scikit-learn da Frente 4 (iot-esp32/ml-edge/model.pkl) quando
    carregado; senão usa as regras determinísticas (_classify_risk_rules).
    """
    if _model_bundle is None:
        return _classify_risk_rules(hr, spo2, temp, radiation, resp)
    values = {"hr": hr, "spo2": spo2, "temp": temp, "radiation": radiation, "resp": resp}
    row = [[values[f] for f in _model_bundle["features"]]] # respeita a ordem do bundle
    try:
        return str(_model_bundle["model"].predict(row)[0])
    except Exception: # qualquer falha de inferência → regra (nunca derruba a API)
        return _classify_risk_rules(hr, spo2, temp, radiation, resp)
```

2.7 Frente 5 — Backend, Dashboard, DevOps e Governança

Além de orquestrar as frentes, esta frente entrega: **governança de IA** (toda consulta ao agente é registrada em uma trilha de auditoria — pergunta, resposta, fontes, canal e timestamp), **DevOps** (Docker + docker-compose sobem tudo com um comando; CI no GitHub Actions roda testes a cada push) e **qualidade** (37 testes automatizados). A telemetria real do ESP32 tem prioridade sobre a simulação no stream em tempo real.

backend/main.py

```
def _log_alert(state: dict, new_risk: str) -> None:
    alert = {
        "ts": _now(),
        "crew_id": state["id"],
        "name": state["name"],
        "risk_level": new_risk,
        "message": (
            f"{state['name']} entrou em estado {new_risk.upper()} "
            f"(HR {state['hr']} bpm · SpO2 {state['spo2']}% · "
            f"Temp {state['temp']}°C · Rad {state['radiation']} µSv/h)"
        ),
    }
    db.insert_alert(alert) # persiste no SQLite (sobrevive a reinícios)
```

2.8 Interfaces desenvolvidas (dashboard)

O centro de controle web reúne, em tema "mission control", a telemetria ao vivo dos três tripulantes (cards com risco + gráficos de frequência cardíaca e radiação) e o Copiloto, com entrada por voz (wake word "Astro") e por texto. Páginas dedicadas trazem a análise de imagem, o log de alertas e a trilha de auditoria.

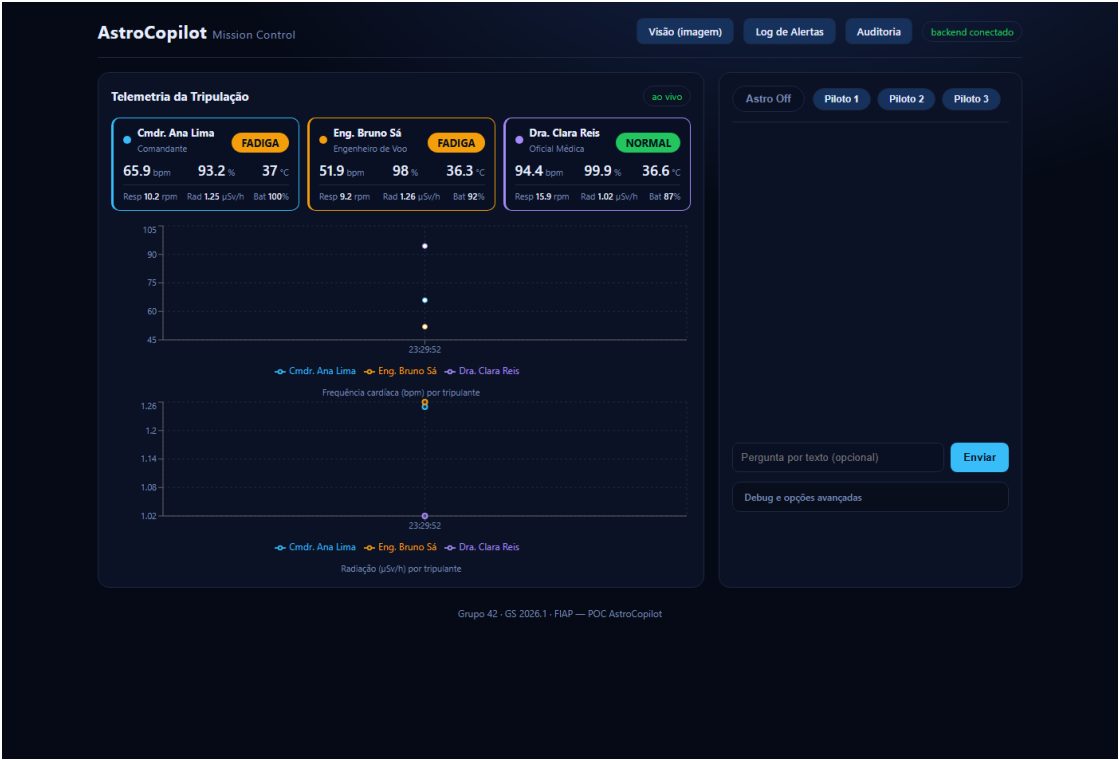


Figura 4 — Dashboard: telemetria ao vivo de 3 tripulantes + Copiloto (RAG).

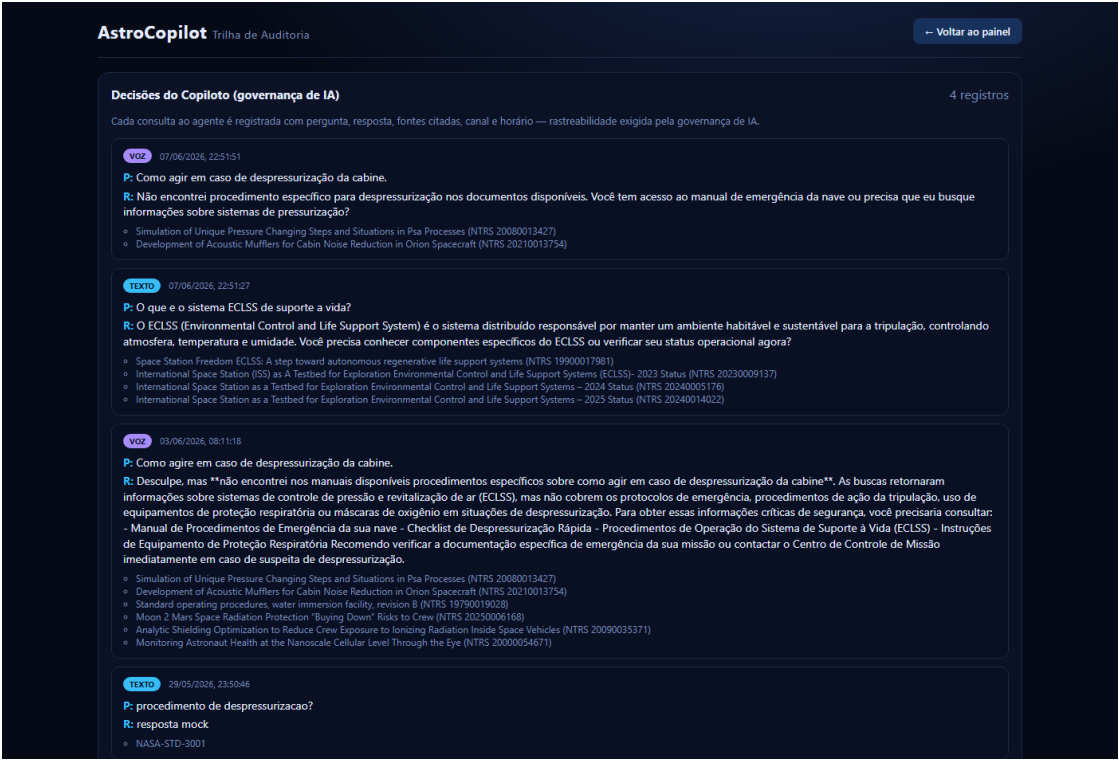


Figura 5 — Trilha de auditoria: cada decisão do agente é registrada (governança).

3. Resultados Esperados

A POC demonstra, ponta a ponta e de forma testada, os seguintes resultados:

- **Respostas fundamentadas:** o copiloto responde perguntas técnicas citando manuais reais da NASA (NTRS), reduzindo alucinação e aumentando a confiabilidade a bordo.
- **Operação sem as mãos / acessível:** a tripulação fala ("Astro, ...") e ouve a resposta, útil em emergências e para tripulantes com limitação visual.
- **Monitoramento vital em tempo real:** sinais de 3 tripulantes em streaming a 1 Hz, com classificação automática de risco e log de alertas persistente.
- **Leitura assistida de painéis:** a câmera identifica componentes e lê displays via OCR.
- **Rastreabilidade (governança):** toda decisão do agente fica auditável, requisito central para IA aplicada a ambientes críticos.
- **Reprodutibilidade:** um único `docker compose up` sobe toda a stack; CI garante que o projeto continua operacional a cada alteração.

O **smoke test** executado sobre a stack integrada validou os 11 fluxos principais (health, tripulação, telemetria POST/GET, alertas, RAG real, auditoria, visão, voz STT→RAG→TTS, TTS e entrega do MP3) — todos com sucesso e sem erros em log.

4. Conclusões

O AstroCopilot responde à pergunta-desafio mostrando, na prática, como IA Generativa, visão computacional, IoT/Edge, Machine Learning e automação se combinam para apoiar a operação na nova economia espacial. A maior contribuição do trabalho foi a **integração real entre as disciplinas das Fases 3 e 4**: cada frente entregou um módulo funcional que conversa com as demais através de um orquestrador comum, em vez de protótipos isolados.

Como evolução natural (fora do escopo desta POC), prevemos: agente com mais ferramentas (acionar visão e telemetria autonomamente), base RAG ampliada com PDFs completos, modelo de visão fine-tuned para componentes específicos da nave, e um aplicativo mobile (React Native) para a equipe de solo. A arquitetura modular adotada torna essas extensões diretas.

5. Perspectivas: Computação Quântica e Neuromórfica

Olhando para o futuro da economia espacial, duas fronteiras computacionais se conectam diretamente ao AstroCopilot. A **computação quântica** promete acelerar problemas hoje intratáveis em escala clássica: otimização de trajetórias e janelas de lançamento, simulação de novos materiais e propelentes (química quântica), e criptografia resistente para comunicações espaço-Terra. Algoritmos como QAOA e Grover poderiam, por exemplo, otimizar o escalonamento de tarefas e recursos energéticos de uma missão — exatamente o tipo de decisão que o copiloto hoje apoia com heurísticas e ML.

Já a **computação neuromórfica** — chips que imitam o funcionamento de neurônios e sinapses, como Intel Loihi e IBM TrueNorth — é especialmente promissora para o **Edge espacial**: oferece inferência de IA com consumo de energia ordens de grandeza menor, processamento orientado a eventos e tolerância a ruído/radiação. Em um wearable como o da Frente 4, um processador neuromórfico permitiria rodar a

classificação de risco (e até modelos de visão) localmente, em tempo real e com bateria mínima, sem depender de conectividade — uma evolução direta do nosso modelo de ML na borda. Ambas as tecnologias reforçam a tese do projeto: levar inteligência confiável e eficiente para o ambiente extremo do espaço.

6. Repositório, Execução e Vídeo

Repositório: <https://github.com/marcofiap/fase4-GS1-AstroCopilot>

Vídeo demonstrativo (YouTube — Não listado): _____ (inserir link antes da entrega)

Como executar

```
# Opção 1 — Docker (sobe backend + dashboard)
docker compose up --build
#   Dashboard: http://localhost:5173
#   Backend:    http://localhost:8000/docs

# Opção 2 — desenvolvimento
cd backend && python -m venv .venv && source .venv/Scripts/activate
pip install -r requirements.txt && uvicorn main:app --reload
cd dashboard && npm install && npm run dev
```

Configuração: um único `.env` na raiz com a chave do Bedrock (`AWS_BEARER_TOKEN_BEDROCK`). O banco SQLite é criado automaticamente. Detalhes completos no README de cada pasta.